1. Executive Summary

This report documents eight weeks of structured learning and hands-on development as part of the Seasons of Code 2025 programme. The project, titled Smart Camera Alert System, aims to build an end-to-end computer-vision pipeline that ingests live or recorded video frames, preprocesses them, runs object detection, and surfaces alerts through a web dashboard backed by a REST API.

By the mid-term checkpoint I have completed foundational modules spanning block-based visual programming, Python scripting, web development, API design, graph-based pipeline modelling, and core computer-vision skills including image arrays, preprocessing chains, and object detection. All source code is version-controlled in the GitHub repository linked above.

---

2. Weekly Progress

Week 1 – Computing Without Fear

Topics Covered

- Scratch event-driven programming model (sprites, costumes, broadcasts)
- Blockly visual blocks and code generation
- Flowchart notation – decision diamonds, process boxes, connectors
- Core CS concepts: variables, loops, events, conditional logic (CS50 Week 0)

Lab Work

Built a Scratch project that simulates a camera alert system at the visual-programming level. The project uses:

- A motion-sensor sprite that fires a broadcast event when it detects movement (simulated via key press)
- An alert sprite subscribed to that broadcast which changes costume and plays a sound
- A variable ALERT_COUNT that increments on each trigger and is displayed on screen

Separately, I drew a camera alert flowchart using draw.io covering the full decision path: frame captured → motion detected? → confidence above threshold? → log alert → notify user. The flowchart is committed to the repo under docs/week1_flowchart.png.

Sample Code / Pseudocode Explored

```
// Scratch pseudocode equivalent
when flag clicked:
  set ALERT_COUNT to 0
when [space] key pressed:
```

```
  broadcast 'motion_detected'
when I receive 'motion_detected':
  change ALERT_COUNT by 1
  play sound 'alert'
  switch costume to 'alarm_on'
```

Homework Summary

Written one-pager on: Variables (named memory locations), Loops (repeat/while – used for continuous frame sampling), Events (asynchronous triggers – camera interrupt), Conditions (if/else – threshold comparison). Committed as docs/week1_homework.md.

Week 2 – Python As A Tool

Topics Covered

- Functions, default arguments, return values
- List comprehensions and filtering
- Dictionaries as structured data containers
- Writing tests with pytest – assertions, fixtures

Lab Work

Built a detection_utils.py module that handles the core data structures used in object detection output:

```
# detection_utils.py
from typing import List, Dict


Detection = Dict  # {label: str, confidence: float, box: list}


def build_detection_dict(label: str, confidence: float, box: list) ->
Detection:
    """Wrap raw detector output into a structured dict."""
    return {
        'label': label,
        'confidence': round(confidence, 4),
        'box': box,          # [x1, y1, x2, y2]
        'area': (box[2]-box[0]) * (box[3]-box[1])
    }


def filter_by_confidence(detections: List[Detection],
                         threshold: float = 0.5) -> List[Detection]:
    """Return only detections above the confidence threshold."""
    return [d for d in detections if d['confidence'] >= threshold]
```

```
def filter_by_label(detections: List[Detection],
                    allowed: List[str]) -> List[Detection]:
    return [d for d in detections if d['label'] in allowed]
```

Homework – Tests

```
# test_detection_utils.py
import pytest
from detection_utils import build_detection_dict, filter_by_confidence

@pytest.fixture
def sample_detections():
    return [
        build_detection_dict('person', 0.92, [10, 20, 80, 200]),
        build_detection_dict('car',    0.45, [50, 60, 300, 400]),
        build_detection_dict('dog',    0.71, [5, 5, 40, 60]),
    ]

def test_filter_keeps_high_confidence(sample_detections):
    result = filter_by_confidence(sample_detections, 0.5)
    assert len(result) == 2
    assert all(d['confidence'] >= 0.5 for d in result)

def test_filter_empty_when_threshold_too_high(sample_detections):
    result = filter_by_confidence(sample_detections, 0.99)
    assert result == []

def test_area_computed_correctly():
    d = build_detection_dict('person', 0.9, [0, 0, 100, 50])
    assert d['area'] == 5000
```

Week 3 – Web Basics and React Thinking

Topics Covered

- HTML5 semantic elements, CSS Flexbox and Grid
- JavaScript ES6+ – arrow functions, destructuring, fetch API
- TypeScript basics – interfaces, type narrowing
- React fundamentals – JSX, props, useState, useEffect

Lab – Three-Stage Dashboard Mock

Built a React TypeScript dashboard with three logical panels: Live Feed (placeholder camera canvas), Detection Panel (scrolling list of alert cards), and Control Panel (threshold slider, label filter, start/stop toggle). State is managed with useState hooks and the threshold filter is wired to the Detection Panel in real time.

```
// DashboardState interface
interface DashboardState {
  isStreaming: boolean;
  confidenceThreshold: number;
  allowedLabels: string[];
  alerts: Alert[];
}

const [state, setState] = useState<DashboardState>({
  isStreaming: false,
  confidenceThreshold: 0.5,
  allowedLabels: ['person', 'car', 'bicycle'],
  alerts: [],
});
```

Homework – UI State Diagram

Submitted a Mermaid state diagram (docs/week3_state_diagram.md) showing transitions between IDLE → STREAMING → ALERT → ACKNOWLEDGED → STREAMING, with guard conditions on each arc.

Week 4 – APIs and JSON Contracts

Topics Covered

- FastAPI routing, path/query parameters, Pydantic schemas
- HTTP verbs (GET/POST/PUT/DELETE) and status codes
- JSON schema validation – required fields, enum values, ranges
- Postman for manual endpoint testing

Lab – Image Analysis API Contract

```
# main.py (FastAPI)
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import List
import uuid, datetime

app = FastAPI(title='Camera Alert API', version='0.1.0')
```

```
class BoundingBox(BaseModel):
    x1: int; y1: int; x2: int; y2: int

class DetectionResult(BaseModel):
    label: str
    confidence: float = Field(..., ge=0.0, le=1.0)
    box: BoundingBox

class AnalyseRequest(BaseModel):
    frame_id: str
    image_base64: str
    confidence_threshold: float = 0.5

class AnalyseResponse(BaseModel):
    request_id: str
    timestamp: str
    detections: List[DetectionResult]
    alert_triggered: bool

@app.post('/analyse', response_model=AnalyseResponse, status_code=200)
async def analyse_frame(req: AnalyseRequest):
    # stub – real model call goes here
    detections = []
    return AnalyseResponse(
        request_id=str(uuid.uuid4()),
        timestamp=datetime.datetime.utcnow().isoformat(),
        detections=detections,
        alert_triggered=False
    )
```

Homework – Request / Response Bodies

Example POST /analyse request:

```
{
  "frame_id": "frame_00421",
  "image_base64": "<base64-encoded-jpeg>",
  "confidence_threshold": 0.6
}
```

Example 200 response:

```
{
  "request_id": "a3f9c2d1-...",
  "timestamp": "2025-06-10T14:32:01.123Z",
  "detections": [
    { "label": "person", "confidence": 0.91,
```

```
      "box": { "x1": 12, "y1": 45, "x2": 98, "y2": 210 } }
  ],
  "alert_triggered": true
}
```

Week 5 – Blocks and Graphs

Topics Covered

- Blockly custom block definition (JSON + JS generator)
- React Flow nodes, edges, and viewport controls
- Graph theory – DAG, topological sort, cycle detection
- NetworkX for programmatic graph validation

Lab – Block Graph and Validation

```
# graph_validator.py
import networkx as nx


def build_pipeline_graph(blocks: list, edges: list) -> nx.DiGraph:
    G = nx.DiGraph()
    G.add_nodes_from([b['id'] for b in blocks])
    for e in edges:
        G.add_edge(e['source'], e['target'])
    return G


def validate_graph(G: nx.DiGraph) -> dict:
    issues = []
    if not nx.is_directed_acyclic_graph(G):
        issues.append('Graph contains a cycle – pipeline would loop forever')
    roots = [n for n in G if G.in_degree(n) == 0]
    if len(roots) != 1:
        issues.append(f'Expected 1 source node, found {len(roots)}')
    return {'valid': len(issues) == 0, 'issues': issues,
            'topo_order': list(nx.topological_sort(G)) if not issues else []}
```

Homework – Graph Validation Checklist

- Graph must be a directed acyclic graph (DAG)
- Exactly one source node (in-degree = 0)
- At least one sink node (out-degree = 0)
- Every node reachable from the source
- No dangling edges (both endpoints must exist as nodes)
- Port type compatibility on each edge (e.g., image → image, not image → text)

Week 6 – Images As Arrays

Topics Covered

- NumPy image arrays – shape (H, W, C), dtype uint8
- BGR vs RGB channel ordering in OpenCV
- Resize, grayscale conversion, channel splitting
- ROI cropping and pixel-value inspection

Lab Work

```
# image_basics.py
import cv2
import numpy as np

img = cv2.imread('test_frame.jpg')          # BGR, shape (H, W, 3)
print(f'Shape: {img.shape}  Dtype: {img.dtype}')

# Resize to fixed input size
resized = cv2.resize(img, (640, 480))

# Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  # shape (H, W)

# Crop a region of interest
roi = img[100:300, 150:400]                  # [y1:y2, x1:x2]

# Pixel stats
print(f'Mean brightness: {gray.mean():.1f}')
print(f'Min: {gray.min()}  Max: {gray.max()}')

cv2.imwrite('gray_output.jpg', gray)
cv2.imwrite('roi_output.jpg', roi)
```

Homework

Submitted before/after image pairs (original colour frame vs grayscale vs resized) with a written explanation noting that grayscale reduces memory from 3× to 1× per pixel and speeds up subsequent operations like edge detection. Committed as docs/week6_images/.

Week 7 – Preprocessing and Frame Efficiency

Topics Covered

- Gaussian and median blur – noise reduction trade-offs
- Adaptive thresholding vs global Otsu thresholding
- Canny edge detection – gradient magnitude and hysteresis
- Temporal subsampling – process every Nth frame
- Perceptual hashing (pHash) for duplicate frame removal

Lab Work

```
# preprocess.py
import cv2, numpy as np
import imagehash, PIL.Image


def preprocess_frame(frame: np.ndarray,
                     blur_ksize: int = 5,
                     low_light: bool = False) -> np.ndarray:
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    if low_light:
        gray = cv2.equalizeHist(gray)    # CLAHE alternative
    blurred = cv2.GaussianBlur(gray, (blur_ksize, blur_ksize), 0)
    _, thresh = cv2.threshold(blurred, 0, 255,
                              cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    edges = cv2.Canny(blurred, 50, 150)
    return edges


def is_duplicate(frame: np.ndarray, prev_hash,
                 threshold: int = 8):
    pil = PIL.Image.fromarray(frame)
    h = imagehash.phash(pil)
    if prev_hash is None:
        return False, h
    return (h - prev_hash) < threshold, h
```

Homework – Low-Light Preprocessing Policy

Policy document (docs/week7_lowlight_policy.md) defines: apply CLAHE (clip limit 2.0, tile 8×8) before blur; increase Gaussian kernel to 7×7 to suppress noise amplified by histogram equalisation; lower Canny low threshold to 30; drop pHash threshold to 5 because low-light frames carry less discriminative information.

Week 8 – Object Detection

Topics Covered

- YOLO architecture overview – anchor boxes, feature pyramid
- Bounding box formats – XYWH vs XYXY, normalised vs absolute
- Confidence score and class probability
- Intersection over Union (IoU) – formula and use in NMS
- Non-Maximum Suppression algorithm
- mAP@0.5 and mAP@0.5:0.95 metrics

Lab Work

```
# detect.py
from ultralytics import YOLO
import cv2

model = YOLO('yolov8n.pt')   # nano – fast inference

def run_detection(frame, conf_threshold=0.5):
    results = model(frame, conf=conf_threshold, verbose=False)
    detections = []
    for r in results:
        for box in r.boxes:
            detections.append({
                'label': model.names[int(box.cls)],
                'confidence': float(box.conf),
                'box': box.xyxy[0].tolist()    # [x1,y1,x2,y2]
            })
    return detections

# Visualise
frame = cv2.imread('test.jpg')
dets = run_detection(frame)
for d in dets:
    x1,y1,x2,y2 = map(int, d['box'])
    cv2.rectangle(frame, (x1,y1),(x2,y2), (0,255,0), 2)
    cv2.putText(frame, f"{d['label']} {d['confidence']:.2f}",
                (x1,y1-8), cv2.FONT_HERSHEY_SIMPLEX, 0.5,(0,255,0),1)
cv2.imwrite('annotated.jpg', frame)
```

Failure Analysis Table

Failure Type	Example Scenario	Root Cause	Mitigation

False negative	Pedestrian at 15 m distance	Object too small for anchor	Tile and stitch input
False positive	Bush flagged as person	Texture similarity	Raise conf threshold to 0.65
IoU miss	Two people overlapping	NMS too aggressive (IoU=0.3)	Increase IoU to 0.5
Low-light miss	Night-time parking lot	Insufficient brightness	Apply CLAHE preprocessing
Occlusion miss	Person half behind car	Partial bounding box	Use pose estimation model

---

3. Key Milestones Achieved

Week	Milestone	Deliverable
1	Event-driven Scratch project complete	scratch_project.sb3 + flowchart
2	Detection dict module with passing tests	detection_utils.py + test_detection_utils.py
3	React dashboard skeleton with state wiring	src/ React app (TypeScript)
4	FastAPI contract live, tested in Postman	main.py + postman_collection.json
5	DAG validator with topological sort	graph_validator.py + NetworkX notebook
6	OpenCV image array pipeline	image_basics.py + before/after images
7	Preprocessing chain with pHash dedup	preprocess.py + policy doc
8	YOLOv8 inference with annotation overlay	detect.py + annotated sample images

---

4. Challenges Faced

4.1 OpenCV BGR vs RGB Confusion

Early in Week 6 I spent an afternoon debugging why my grayscale images had a blue tint when converted via PIL. The issue was that OpenCV loads images in BGR order while PIL and matplotlib expect RGB. I added a cv2.cvtColor(img, cv2.COLOR_BGR2RGB) conversion step at every display boundary and documented this as a project-wide convention.

4.2 YOLO Inference Speed on CPU

Running YOLOv8n on a laptop CPU gave ~3–4 FPS on 640×480 input, too slow for real-time alerting. I resolved this in the lab by enabling half-precision inference (model.predict(..., half=True)) and batching frames in groups of 4, achieving ~10 FPS. The long-term plan is to switch to GPU inference or export to ONNX + OpenVINO.

4.3 Graph Cycle Detection Edge Case

The NetworkX is_directed_acyclic_graph check passed on my first test graph even though I had a self-loop on a 'Resize' node. This was because I accidentally added the node but never connected the self-edge. I caught the bug by writing an explicit unit test that adds a self-loop and expects the validator to fail.

4.4 Pydantic v1 vs v2 API Differences

My FastAPI code was written following v1 examples (BaseModel with .dict()) but the environment had Pydantic v2 installed where the method is .model_dump(). I updated all serialisation calls and pinned pydantic==2.7.1 in requirements.txt.

---

5. Plan for Remaining Weeks

- Week 9: Connect React dashboard to live FastAPI WebSocket stream
- Week 10: Integrate full preprocessing → detection pipeline end-to-end
- Week 11: Add alert logging with SQLite and review history UI
- Week 12: Write system tests, tune thresholds, prepare final demo